

NEXFLEET

TDE 2 • BACKEND

Vertical Slice, Clean Architecture e SOLID

Documentação da Parte 2 e implementação executável

ALUNOS

Lucas Mockel Roussenq

Vinícius Reis

Erik Carvalho

Mateus Burlamaqui

Branch: feature/vertical-slice-clean-solid

8 de outubro de 2026

1. Visão executiva

Este documento registra a evolução da Parte 2 do NexFleet e a implementação do primeiro recorte executável do backend. A solução foi organizada como monólito modular, usando Vertical Slice para agrupar cada caso de uso, Clean Architecture para controlar a direção das dependências e SOLID para manter responsabilidades pequenas, contratos explícitos e infraestrutura substituível.

RESULTADO ENTREGUE

Duas fatias funcionais — autenticação por JWT e criação de usuário por administrador — com PostgreSQL, Flyway, BCrypt, tratamento de erros HTTP, testes unitários e regras ArchUnit. A suíte final executada no Java 21 contém 12 testes aprovados, sem falhas, erros ou testes ignorados.

Escopo implementado

- Login com e-mail normalizado e senha protegida por BCrypt.
- Emissão e validação de JWT em sessão stateless.
- Criação de usuários restrita ao perfil ADMINISTRADOR.
- Administrador inicial configurado por variáveis de ambiente, sem senha fixa no repositório.
- Persistência PostgreSQL com migração Flyway e e-mail único.
- Contratos HTTP separados das entidades de domínio e das entidades JPA.
- Testes de regras e testes automatizados de arquitetura.

Decisão de escopo

O recorte corresponde à Sprint 1. Veículos, motoristas, abastecimentos, manutenções e dashboard foram mantidos fora desta entrega porque dependem de regras de negócio adicionais. A decisão evita antecipar estruturas ainda não validadas e permite demonstrar as três abordagens arquiteturais com código testável de ponta a ponta.

2. Participação dos alunos

A divisão abaixo registra as responsabilidades assumidas nesta entrega. O grupo participou da revisão final do código, dos testes e da consistência entre implementação e documentação.

Aluno	Atividade realizada
Lucas Mockel Roussenq	Modelagem da entidade Usuario, validações de domínio e revisão da aplicação de SRP e OCP.
Vinícius Reis	Definição dos contratos REST, fluxo de login e controle de autorização com JWT.
Erik Carvalho	Persistência PostgreSQL/Flyway, adapters JPA e testes automatizados de arquitetura com ArchUnit.
Mateus Burlamaqui	Documentação da Parte 2, diagramas de classes/componentes e registro dos prompts utilizados.

OBSERVAÇÃO

A distribuição deve ser ajustada pelo grupo caso a participação real tenha ocorrido de forma diferente da registrada.

3. Arquitetura da Parte 2

Contexto

O NexFleet é uma aplicação de gestão de frota. A arquitetura planejada na Parte 2 foi refinada nesta entrega para uma forma executável: um backend Spring Boot com módulos funcionais, banco PostgreSQL e autenticação JWT. O monólito modular reduz custo operacional no estágio atual sem impedir uma separação futura de módulos.

Objetivos arquiteturais

- Manter regras de domínio e casos de uso independentes de Spring, JPA e transporte HTTP.
- Permitir que cada mudança de negócio seja localizada em uma fatia vertical.
- Fazer a infraestrutura depender de portas internas, preservando a inversão de dependência.
- Impedir ciclos entre módulos e dependências acidentais do domínio em frameworks.

Restrições e premissas

- Java 21 e Maven Wrapper para build reproduzível.
- Spring Boot, Spring Security, Spring Data JPA e Flyway como detalhes externos ao núcleo.
- PostgreSQL como persistência relacional e Docker Compose para execução local.
- Segredos fornecidos por ambiente; nenhum segredo real é versionado.

Decisões adiadas

Microserviços, event-driven architecture, API Gateway, BFF, serverless e microfrontends permanecem como opções futuras. Nesta etapa, adicioná-los aumentaria a complexidade sem evidência de escala, autonomia de equipes ou necessidade operacional que justificasse a mudança.

4. Vertical Slice na implementação

Vertical Slice organiza o código pelo comportamento entregue ao usuário, em vez de criar grandes camadas horizontais compartilhadas. Cada fatia contém entrada HTTP, comando, caso de uso, portas e testes necessários para concluir aquele fluxo.

Fatia 1 — autenticar usuário

- Entrada: LoginController e LoginRequest.
- Aplicação: LoginUseCase, LoginService e LoginCommand.
- Portas: BuscarUsuarioPorEmail, PasswordVerifier e TokenIssuer.
- Adapters: repositório JPA, verificador BCrypt e emissor JWT.
- Saída: LoginResponse com token e metadados seguros.

Fatia 2 — criar usuário

- Entrada: CriarUsuarioController e CriarUsuarioRequest.
- Aplicação: CriarUsuarioUseCase, CriarUsuarioService e comando próprio.
- Domínio: fábrica e invariantes da entidade Usuario.
- Portas: UsuarioRepository e PasswordHasher.
- Saída: DTO sem hash de senha e status HTTP 201.

Benefício observado

Uma alteração no login não exige navegar por pacotes globais de controller, service e repository. O fluxo está coeso em auth/login; a criação de usuários está coesa em usuarios/criar. Compartilhamentos só aparecem quando representam contrato estável, como o domínio de usuário ou a API pública de consulta por e-mail.

5. Clean Architecture

A regra central é a direção das dependências: detalhes externos conhecem contratos internos; o núcleo não conhece frameworks. No NexFleet, controllers convertem HTTP em comandos, serviços de aplicação orquestram regras, o domínio preserva invariantes e adapters implementam portas.

Mapeamento das camadas

Anel	Responsabilidade	Exemplos
Domínio	Estado e regras essenciais	Usuario, Perfil
Aplicação	Casos de uso e portas	LoginService, CriarUsuarioService, interfaces
Adapters	Tradução entre núcleo e tecnologia	Controllers, JPA adapters, BCrypt, JWT
Frameworks	Runtime e mecanismos externos	Spring Boot, PostgreSQL, Flyway

Regra de dependência

A sequência principal é `api` → `application` → `domain`. A infraestrutura não é chamada diretamente pelo caso de uso: ela implementa interfaces definidas no lado interno. Essa inversão permite testar os serviços com dublês simples e trocar tecnologias sem reescrever regras.

Composição

`UseCaseConfig` é o composition root: instancia serviços de aplicação com implementações de portas fornecidas pelo Spring. `SecurityConfig` concentra a política de autenticação e autorização. Essas classes pertencem ao limite externo e podem conhecer os dois lados para montar a aplicação.

6. SOLID aplicado

Princípio	Aplicação no NexFleet
S — Responsabilidade única	Controllers tratam HTTP; casos de uso orquestram; Usuario preserva invariantes; adapters tratam detalhes técnicos.
O — Aberto/fechado	Novos mecanismos de token, hash ou persistência entram como implementações de portas, sem alterar o fluxo central.
L — Substituição de Liskov	Adapters respeitam os contratos de PasswordHasher, PasswordVerifier, TokenIssuer e repositórios, permitindo substituição previsível.
I — Segregação de interfaces	Interfaces são pequenas e orientadas ao caso de uso: buscar usuário, verificar senha, emitir token, persistir usuário.
D — Inversão de dependência	LoginService e CriarUsuarioService dependem de abstrações internas; JPA, BCrypt e JWT dependem dessas abstrações.

USO PRAGMÁTICO

SOLID foi usado para reduzir acoplamento onde existe uma fronteira real. Não foram criadas abstrações sem função: registros de comando/resultado e DTOs permanecem simples, e o compartilhamento é limitado a contratos estáveis.

7. Estrutura do backend

A estrutura combina módulo funcional, fatia vertical e separação interna de responsabilidades:

```
backend/src/main/java/br/com/nexfleet
├── auth/login
│   ├── api
│   └── application
├── auth/infrastructure/security
├── usuarios/criar
│   ├── api
│   └── application
├── usuarios/domain
├── usuarios/publicapi
├── usuarios/infrastructure
└── shared
```

Regras automatizadas

- O domínio não depende de classes do Spring ou de Jakarta Persistence.
- Pacotes de API não são acessados por domínio ou aplicação.
- A aplicação não depende de infraestrutura.
- Módulos de alto nível permanecem livres de ciclos de dependência.

Contrato HTTP

```
POST /api/v1/auth/login
{ "email": "admin@nexfleet.local", "senha": "..."}

POST /api/v1/usuarios (Bearer JWT + ADMINISTRADOR)
{ "nome": "Gestor da Frota", "email": "gestor@nexfleet.local",
  "senha": "senha-com-8-ou-mais-caracteres", "perfil": "GESTOR_FROTA" }
```

8. Diagrama de classes do backend

O diagrama representa as classes centrais das duas fatias. Interfaces são portas; linhas tracejadas indicam implementação e setas contínuas representam dependência de uso.

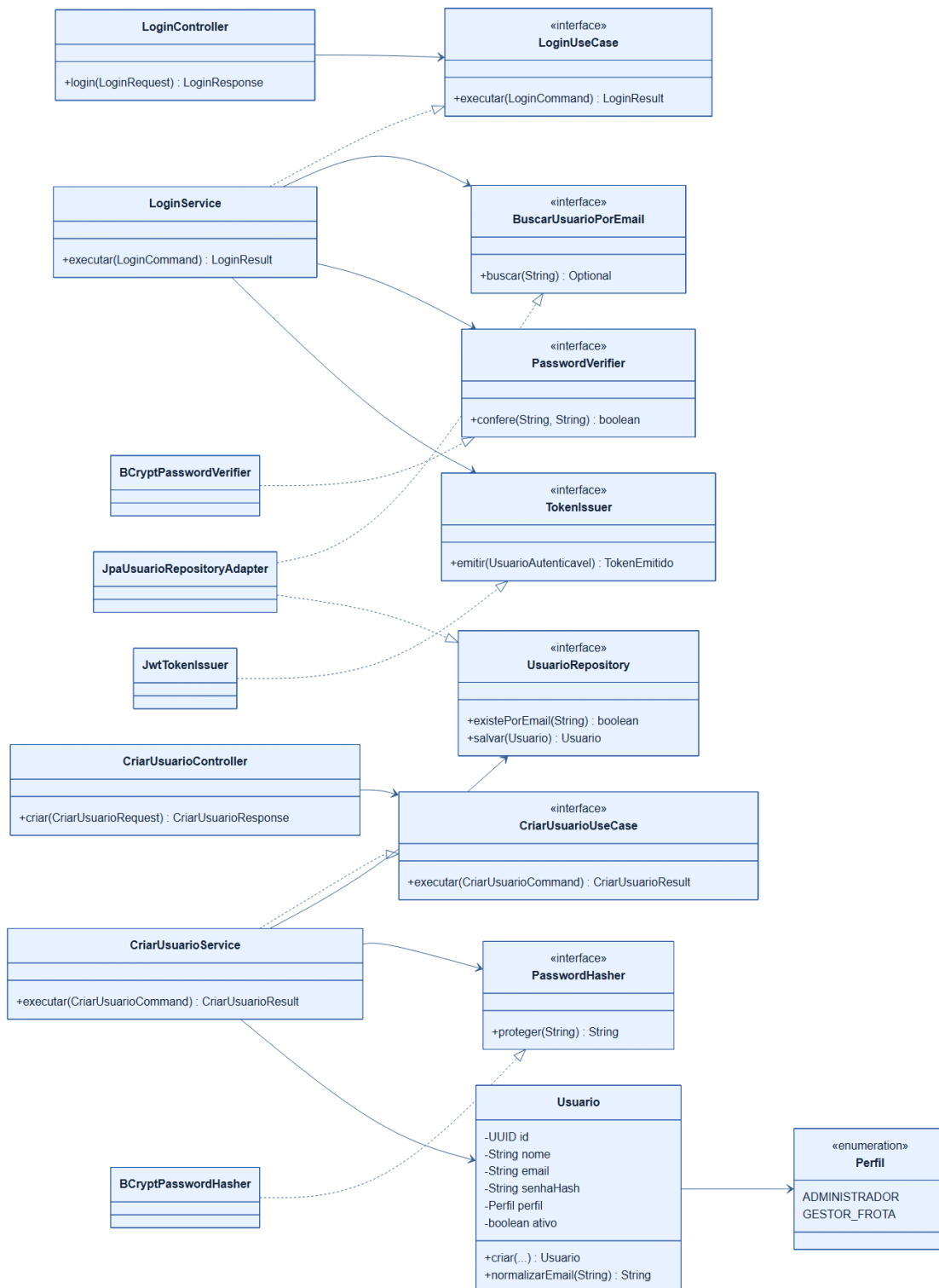


Figura 1 — Classes das fatias de login e criação de usuário.

9. Leitura do diagrama de classes

Autenticação

LoginController depende de LoginUseCase. LoginService implementa esse caso de uso e recebe três capacidades: localizar o usuário, verificar a senha e emitir o token. Nenhuma dessas capacidades exige que o serviço conheça JPA, BCrypt ou a biblioteca JWT.

Criação de usuário

CriarUsuarioController chama CriarUsuarioUseCase. O serviço verifica unicidade, solicita o hash da senha, cria a entidade de domínio e a salva por UsuarioRepository. A resposta HTTP é construída a partir do resultado do caso de uso, sem devolver o hash.

Domínio e adapters

Usuario concentra normalização e invariantes. JpaUsuarioRepositoryAdapter implementa os contratos de escrita e consulta; adapters BCrypt implementam hash/verificação; JwtTokenIssuer implementa emissão do token. Essa disposição evidencia ISP e DIP.

Fontes do diagrama

- Markdown/Mermaid: docs/diagramas/diagrama-classes-backend.md.
- Fonte renderizável: docs/diagramas/diagrama-classes-backend.mmd.
- Imagem: docs/diagramas/diagrama-classes-backend.png.

10. Diagrama de componentes

O diagrama mostra o fluxo entre entrada HTTP, casos de uso, núcleo e infraestrutura. O PostgreSQL permanece atrás do adapter JPA e da migração Flyway.

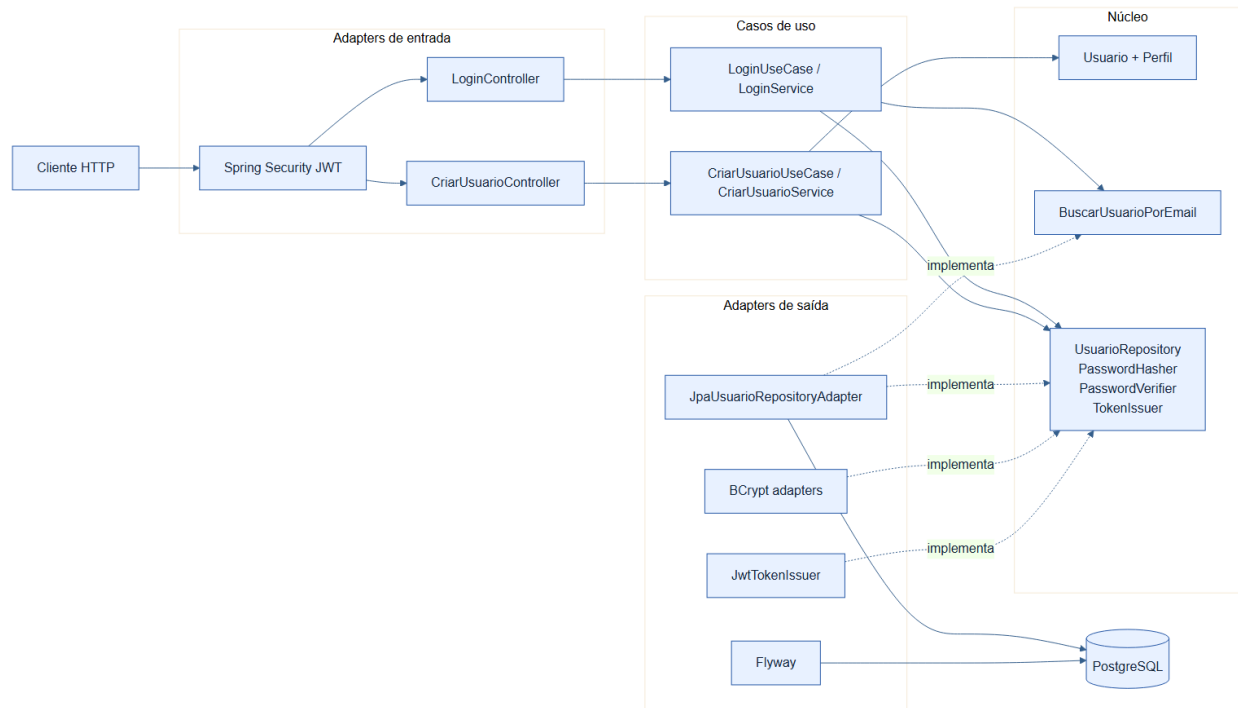


Figura 2 — Componentes e dependências do backend.

DIREÇÃO

O fluxo de execução vai da borda para o núcleo e retorna. A dependência de código permanece apontando para os contratos internos: adapters implementam portas, e casos de uso não importam infraestrutura.

11. Segurança e persistência

Segurança

- Sessão stateless; o backend não mantém sessão de usuário no servidor.
- Senha persistida apenas como hash BCrypt.
- JWT assinado com segredo externo, emissor e tempo de expiração configuráveis.
- Endpoint de criação protegido pelo papel ADMINISTRADOR.
- Credenciais inválidas retornam erro genérico, reduzindo enumeração de usuários.

Persistência

- Tabela usuarios criada por migração versionada V1.
- Identificador UUID e restrição de unicidade no e-mail normalizado.
- Entidade JPA separada da entidade de domínio.
- Adapter realiza o mapeamento entre persistência e domínio.

Configuração segura

```
NEXFLEET_DB_URL=jdbc:postgresql://postgres:5432/nexfleet
NEXFLEET_DB_USER=nexfleet
NEXFLEET_DB_PASSWORD=<definir fora do Git>
NEXFLEET_JWT_SECRET=<mínimo de 32 caracteres>
NEXFLEET_ADMIN_EMAIL=admin@nexfleet.local
NEXFLEET_ADMIN_PASSWORD=<definir fora do Git>
```

12. Evidências de qualidade

BUILD FINAL

Java 21.0.12.1 • 12 testes executados • 0 falhas • 0 erros • 0 ignorados • build aprovado

Conjunto	Quantidade	Cobertura
ArquiteturaBackendTest	4	Dependências, frameworks no domínio e ciclos
LoginServiceTest	3	Sucesso, usuário inativo e credenciais inválidas
CriarUsuarioServiceTest	3	Criação, e-mail duplicado e hash
UsuarioTest	2	Invariantes e normalização

Comando de verificação

```
cd backend
.\mvnw.cmd test
```

O Maven Wrapper fixa o mecanismo de build no próprio projeto. A execução final foi repetida usando explicitamente o Java 21, que é a versão declarada no projeto.

13. GitHub e branch da entrega

Repositório

GITHUB

<https://github.com/vinigoezz/cleancode/tree/feature/vertical-slice-clean-solid>

Branch nova

feature/vertical-slice-clean-solid

Histórico local preparado

80ec797 chore: initialize NexFleet repository
f1ce224 feat: apply vertical slice clean architecture and SOLID

Execução local

- Copiar .env.example para .env e substituir todas as senhas.
- Executar docker compose up --build na raiz do projeto.
- Verificar GET <http://localhost:8080/actuator/health>.
- Autenticar em POST [/api/v1/auth/login](#).
- Usar o token Bearer em POST [/api/v1/usuarios](#).

14. Prompts utilizados

Prompt 1 — solicitação principal

Utilizar IA Generativa para aplicar o VERTICAL SLICE e CLEAN ARCHITECTURE e SOLID no backend da aplicação TDE 2. Utilizar o código da aplicação que está sendo criada no TDE 2. Em documento PDF entregar: no arquivo deve constar o nome dos ALUNOS que auxiliou na tarefa, independente de ser atividade em grupo. (Descreva o que cada aluno realizou). Informar o GITHUB do projeto em uma nova branch. Informar todos os prompts utilizados para modificar a aplicação. Entregar diagrama de classes e componentes do BACKEND da aplicação em VERTICAL SLICE e CLEAN ARCHITECTURE e SOLID (gerar em markdown e imagem). Alunos: Lucas Mockel Roussenq, Vinícius Reis, Erik Carvalho e Mateus Burlamaqui.

Prompt 2 — contexto do projeto

codex://threads/01a0cc7d-99d1-7173-9b7c-1bd536ce7725. Se trata do link acima. Vamos fazer a documentação da parte 2 e desenvolver o trabalho de agora.

Os prompts seguintes registram as instruções de trabalho usadas para decompor a solicitação. Credenciais, segredos e instruções internas da ferramenta não são incluídos.

15. Prompts técnicos derivados

3 — análise do código

Analise a fundação do backend NexFleet e identifique o menor fluxo executável que demonstre Vertical Slice, Clean Architecture e SOLID sem antecipar módulos dependentes.

4 — implementação

Implemente a Sprint 1 com login JWT e criação de usuários por administrador, mantendo domínio e aplicação independentes de Spring/JPA.

5 — segurança

Use BCrypt para senhas, JWT stateless, administrador inicial configurado por ambiente e autorização do endpoint de usuários no backend.

6 — persistência

Implemente PostgreSQL e Flyway com e-mail normalizado e único, sem expor entidades JPA como resposta HTTP.

7 — qualidade

Crie testes unitários dos casos de uso e testes ArchUnit para dependências, frameworks no domínio e ciclos entre módulos.

8 — diagramas

Gere diagramas de classes e componentes que correspondam às classes e dependências realmente entregues, em Mermaid/Markdown e PNG.

9 — documentação

Atualize a Parte 2 para refletir o código entregue, registre SOLID, limitações, evidências de teste, alunos e branch GitHub.

16. Limitações e próximos passos

Limitações conhecidas

- A API ainda não possui documentação OpenAPI publicada.
- Não há rotação/renovação de tokens nem revogação centralizada.
- Os testes desta entrega validam domínio, aplicação e arquitetura; testes de integração com PostgreSQL ficam para o próximo incremento.
- Observabilidade está limitada ao Actuator; métricas e rastreamento distribuído não foram adicionados.

Próximas fatias

- Cadastro de veículos e motoristas com seus próprios casos de uso.
- Registro de abastecimento com validação de vínculo entre veículo e motorista.
- Manutenções, alertas e dashboard como projeções orientadas aos requisitos.
- Testcontainers para integração real com PostgreSQL e Flyway.
- OpenAPI e pipeline de integração contínua executando testes e análise arquitetural.

Critério para evoluir a arquitetura

A separação em serviços independentes só deve ocorrer quando houver necessidade comprovada de escala, autonomia de implantação, isolamento de falhas ou propriedade por equipes distintas. Até lá, o monólito modular preserva simplicidade operacional e fronteiras explícitas.

17. Referências e artefatos

Materiais da disciplina

- SOLID — Repaired.pptx, material fornecido para a atividade.
- VERTICAL SLICE.pptx, material fornecido para a atividade.
- Clean Architecture.pptx, material fornecido para a atividade.

Referências conceituais

- MARTIN, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design.
- MARTIN, Robert C. Agile Software Development, Principles, Patterns, and Practices.

Artefatos entregues no repositório

- docs/ARQUITETURA_PARTE2.md — documentação textual da arquitetura.
- docs/CONTRIBUICOES.md — participação dos alunos.
- docs/PROMPTS_UTILIZADOS.md — registro dos prompts.
- docs/diagramas — Markdown, fontes Mermaid e imagens PNG.
- backend — aplicação, migração e testes.

CONCLUSÃO

A entrega transforma a arquitetura planejada na Parte 2 em um recorte executável e verificável. Vertical Slice define a unidade de mudança, Clean Architecture controla as dependências e SOLID mantém contratos coesos. Os testes confirmam que essas fronteiras estão presentes no código entregue.